

tf.compat.v1.variable_scope Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/compat/v1/variable_scope

A context manager for defining ops that creates variables (layers).

Function Signatures

```
tf.compat.v1.variable_scope( name_or_scope,  
    default_name=None, values=None, initializer=None,  
    regularizer=None, caching_device=None, partitioner=None,  
    custom_getter=None, reuse=None, dtype=None,  
    use_resource=None, constraint=None, auxiliary_name_scope=True  
)
```

Code Examples

```
tf.compat.v1.variable_scope(  
    name_or_scope,  
    default_name=None,  
    values=None,  
    initializer=None,  
    regularizer=None,  
    caching_device=None,  
    partitioner=None,  
    custom_getter=None,  
    reuse=None,  
    dtype=None,  
    use_resource=None,  
    constraint=None,  
    auxiliary_name_scope=True  
)
```

```
tf.compat.v1.variable_scope(  
    name_or_scope,  
    default_name=None,  
    values=None,  
    initializer=None,  
    regularizer=None,  
    caching_device=None,  
    partitioner=None,  
    custom_getter=None,  
    reuse=None,  
    dtype=None,  
    use_resource=None,  
    constraint=None,  
    auxiliary_name_scope=True  
)
```

```
tf.compat.v1.disable_eager_execution()
```

```
tf.compat.v1.disable_eager_execution()
```

```
with tf.compat.v1.variable_scope("foo"):
with tf.compat.v1.variable_scope("bar"):
v = tf.compat.v1.get_variable("v", [1])
assert v.name == "foo/bar/v:0"
```

```
with tf.compat.v1.variable_scope("foo"):
with tf.compat.v1.variable_scope("bar"):
v = tf.compat.v1.get_variable("v", [1])
assert v.name == "foo/bar/v:0"
```

```
with tf.compat.v1.variable_scope("foo") as vs:
pass
# Re-enter the variable scope.
with tf.compat.v1.variable_scope(vs,
auxiliary_name_scope=False) as vs1:
# Restore the original name_scope.
with tf.name_scope(vs1.original_name_scope):
v = tf.compat.v1.get_variable("v", [1])
assert v.name == "foo/v:0"
c = tf.constant([1], name="c")
assert c.name == "foo/c:0"
```

```
with tf.compat.v1.variable_scope("foo") as vs:
pass
# Re-enter the variable scope.
with tf.compat.v1.variable_scope(vs,
auxiliary_name_scope=False) as vs1:
# Restore the original name_scope.
with tf.name_scope(vs1.original_name_scope):
v = tf.compat.v1.get_variable("v", [1])
assert v.name == "foo/v:0"
c = tf.constant([1], name="c")
assert c.name == "foo/c:0"
```

Documentation

A context manager for defining ops that creates variables (layers).

Migrate to TF2

Although it is a legacy `compat.v1` api,
`tf.compat.v1.variable_scope` is mostly compatible with eager
execution and `tf.function` as long as you combine it with the
`tf.compat.v1.keras.utils.track_tf1_style_variables` decorator (though
it will behave as if `reuse` is always set to `AUTO_REUSE`.)

See the
model migration guide
for more info on
migrating code that relies on `variable_scope`-based variable reuse.

When you use it with eager execution enabled but without
`tf.compat.v1.keras.utils.track_tf1_style_variables`,
`tf.compat.v1.variable_scope` will still be able to prefix the names
of variables created within the scope but it will not enable variable reuse
or error-raising checks around variable reuse (`get_variable` calls within
it would always create new variables).

Once you have switched away from `get_variable`-based variable reuse
mechanisms, to switch to TF2 APIs you can just use
`tf.name_scope` to prefix variable names.

Description

Used in the notebooks

- Use TF1.x models in TF2 workflows
- Migrating model checkpoints
- Linear Mixed-Effect Regression in {TF Probability, R, Stan}

- Graph-based Neural Structured Learning in TFX

This context manager validates that the (optional) values are from the same graph, ensures that graph is the default graph, and pushes a name scope and a variable scope.

If `name_or_scope` is not `None`, it is used as is. If `name_or_scope` is `None`, then `default_name` is used. In that case, if the same name has been previously used in the same scope, it will be made unique by appending `_N` to it.

Variable scope allows you to create new variables and to share already created ones while providing checks to not create or share by accident. For details, see the Variable Scope How To, [here](#) we present only a few basic examples.

The Variable Scope works as expected when the Eager Execution is Disabled.

Simple example of how to create a new variable:

Simple example of how to reenter a premade variable scope safely:

Keep in mind that the counters for `default_name` are discarded once the parent scope is exited. Therefore when the code re-enters the scope (for instance by saving it), all nested `default_name` counters will be restarted.

For instance:

Basic example of sharing a variable `AUTO_REUSE`:

Basic example of sharing a variable with `reuse=True`:

Sharing a variable by capturing a scope and setting reuse:

To prevent accidental sharing of variables, we raise an exception when getting an existing variable in a non-reusing scope.

Similarly, we raise an exception when trying to get a variable that does not exist in reuse mode.

Note that the reuse flag is inherited: if we open a reusing scope, then all its sub-scopes become reusing as well.

A note about name scoping: Setting reuse does not impact the naming of other ops such as mult. See related discussion on [github#6189](#)

Note that up to and including version 1.0, it was allowed (though explicitly discouraged) to pass False to the reuse argument, yielding undocumented behaviour slightly different from None. Starting at 1.1.0 passing None and False as reuse has exactly the same effect.

A note about using variable scopes in multi-threaded environment: Variable scopes are thread local, so one thread will not see another thread's current scope. Also, when using default_name, unique scopes names are also generated only on a per thread basis. If the same name was used within a different thread, that doesn't prevent a new thread from creating the same scope. However, the underlying variable store is shared across threads (within the same graph). As such, if another thread tries to create a new variable with the same name as a variable created by a previous thread, it will fail unless reuse is True.

Further, each thread starts with an empty variable scope. So if you wish to preserve name prefixes from a scope from the main thread, you should capture the main thread's scope and re-enter it in each thread. For e.g.

Raises

`## Methods`

`__enter__`

[View source](#)

`__exit__`

[View source](#)